

HOW CAN EVERY ROLLOUT COUNT FOR SKILL SEEKERS?

COMPASS FOR SKILL PROPOSERS

Anonymous authors

Paper under double-blind review

ABSTRACT

Textual-program optimizers use execution traces to revise prompts, instructions, and agent skills, which we collectively call textual programs. A fixed rollout budget must support both candidate revision and the choice of where to search next. COMPASS uses execution results to choose which candidates to revise and which batches to use for those revisions. It compares candidates on shared instances and updates their relative credit as new results arrive. It selects the highest-credit candidates for revision and assigns their proposal batches by maximum-weight matching using recorded score gaps. The method tracks the examples used to create each candidate and its ancestors. Each fixed revision is evaluated on a separately sampled batch outside that history. Proposal generation and admission evaluation share the rollout budget. Qwen3-8B held-out evaluations span mathematical reasoning (e.g., AIME-2025), instruction following (IFBench), multi-hop question answering and fact verification (HotPotQA and HoVer), and privacy-conscious delegation (PUPA). Relative to published GEPA, the largest gain is 36.7 percentage points on AIME-2025, and the mean gain across all six shared benchmarks is 11.8 points.

1 INTRODUCTION

Feedback-driven optimizers revise instructions, prompts, and agent skills from execution traces. Reflection, textual gradients, and evolutionary prompt search use language feedback to produce large behavioral updates with fixed model weights (Shinn et al., 2023; Madaan et al., 2023; Yuksekgonul et al., 2025; Cheng et al., 2024; Agrawal et al., 2025). Given a seed program, a benchmark evaluator, and rollout budget B , an optimizer generates and evaluates candidates, then returns one program. Rollouts provide traces for proposing revisions and scores for comparing candidates. Admission uses a separate batch to decide whether a fixed revision enters the candidate pool. For example, three instruction-following cases may expose a constraint-ordering error and guide a new rule. The rule is then evaluated on other cases.

Each proposal-and-evaluation attempt spends rollout budget on traces and admission scores. An observation can also contribute to later candidate comparisons and proposal decisions. Tracking which examples shaped each candidate determines how that observation can be reused along the lineage.

The central question is how to use a growing, uneven record of execution results to choose which candidates to revise and which batches to use next. We call this *provenance-constrained evidence routing*: proposal history determines which observations can evaluate a candidate, and the recorded results guide the next search decisions. The aim is to maximize the final program’s expected benchmark utility under the available rollout budget.

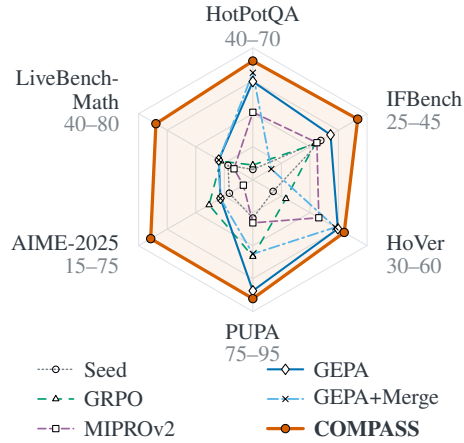


Figure 1: **COMPASS leads on all six tasks.** Qwen3-8B scores (%) use labeled center-to-rim ranges (Table 1).

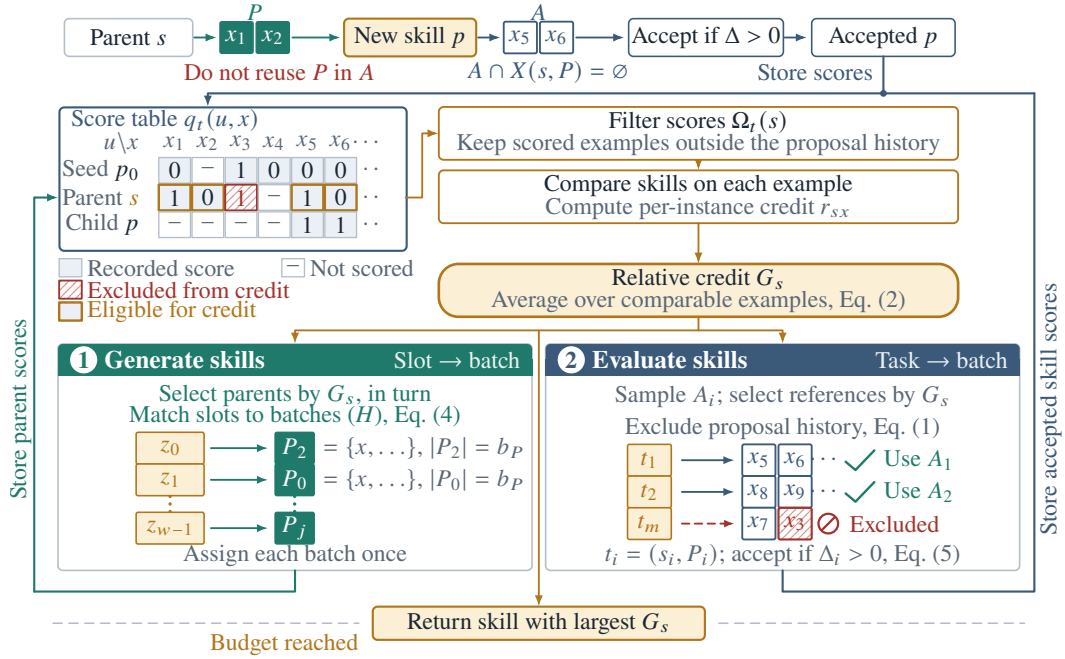


Figure 2: **Candidate comparisons guide the next search wave in COMPASS.** Child p , proposed from parent s on P , is evaluated on a separate legal batch A and enters the cache after admission. Legal same-instance comparisons produce credit G_s , which selects proposal parents, admission references, and the final program. H matches parent slots to proposal batches, while each admission batch is sampled independently outside $X(s, P)$. Red hatching marks lineage-excluded observations.

This requires three linked decisions. First, a candidate’s proposal history must follow its descendants when choosing admission examples. Second, scores collected on different subsets need to be compared on shared instances. The *frontier* on an instance consists of its highest-scoring eligible candidates and changes as results arrive. Third, concurrent proposal attempts need batches matched to each parent’s recorded score gaps.

COMPASS (Credit-Aware Optimization with Mini-batch Proposal and Admission for Skill Search) links these decisions. Recursive exclusion separates proposal and admission examples throughout the lineage. Relative credit updates candidate rankings as instance frontiers change. COMPASS selects the highest-credit candidates for revision, including cutoff ties. Maximum-weight matching assigns batches to their proposal slots, each representing one revision attempt; each fixed child receives a separately sampled legal admission batch. Figure 2 follows this process from candidate generation to final selection.

Completed benchmark evaluations span model families and task types. On Qwen3-8B, the programs selected by COMPASS for each task reach 68.67% on AIME-2025, 43.37% on IFBench, 73.96% on LiveBench-Math, and 54.00% on HoVer. These scores exceed published GEPA by 36.67, 4.76, 22.01, and 1.67 percentage points, respectively.

This paper makes four contributions:

1. *Provenance-constrained evidence routing* formulates fixed-budget search and final selection with proposal history tracked throughout each lineage (Section 2).
2. COMPASS converts eligible same-instance comparisons into relative credit and updates that credit when new results arrive (Sections 3.1 and 3.2).
3. Relative credit selects proposal parents, and maximum-weight matching assigns their batches using recorded score gaps. Paired admission evaluates each resulting revision (Section 3.3).
4. Evaluations under fixed published budgets cover two models and six tasks. COMPASS has the highest six-task aggregate in both model comparisons (Section 5).

2 PROVENANCE-CONSTRAINED EVIDENCE ROUTING

2.1 BUDGETED TEXTUAL-PROGRAM OPTIMIZATION

Given a seed textual program p_0 , a benchmark evaluator, and rollout budget B , a routing policy π adaptively generates and evaluates candidates and returns a final program $p_{\hat{s}_\pi}$, where $\hat{s}_\pi$ is the selected candidate index. Let $R(p)$ denote expected benchmark utility and let R^* denote the best reachable utility under the search model. The budgeted objective is

$$\pi^* \in \arg \min_{\pi \in \Pi(B, b_P, b_A)} \mathbb{E}[R^* - R(p_{\hat{s}_\pi})],$$

where $b_P, b_A \in \mathbb{N}_+$, with $|P| = b_P$ proposal observations and $|A| = b_A$ admission observations per attempt. For policies in $\Pi(B, b_P, b_A)$, B is the wave-level stopping threshold. A new wave starts while the cumulative proposal and evaluation costs satisfy

$$C_{\text{proposal}} + C_{\text{evaluation}} < B,$$

and runs to completion. Let $M_B(\pi)$ denote the number of completed proposal-and-evaluation attempts. Evaluation expenditure assigns b_A admission observations to each fixed child. Proposal generation and admission evaluation consume the shared rollout budget.

At optimization step t , let $\mathcal{P}_t = \{p_0, \dots, p_{m_t-1}\}$ contain the accepted textual programs. Every candidate after p_0 records a parent, inducing a directed acyclic lineage. The *benchmark evaluator* runs each program using the benchmark’s official data, execution procedure, scoring metric, and aggregation rule. Executing program p_s on optimization instance x yields a bounded reward in $[0, 1]$, an output, and an execution trace. The committed cache is a partial table $q_t(s, x)$ containing the highest valid, finite score observed for each evaluated pair, with unevaluated pairs represented by missing entries. A new observation replaces an existing score and its associated output only when its score is strictly higher. Thus each candidate–instance pair retains one record. Let P_a^{birth} be the proposal batch that created candidate a , with $P_0^{\text{birth}} = \emptyset$, and let $\text{Anc}(s)$ contain the transitive ancestors of s . The proposal history of candidate s is

$$\mathcal{B}_s = \bigcup_{a \in \text{Anc}(s) \cup \{s\}} P_a^{\text{birth}}$$

The credit-eligible evidence set is $\Omega_t(s) = \{x : q_t(s, x) \text{ exists and } x \notin \mathcal{B}_s\}$. Relative credit for candidate s therefore draws exclusively from observations outside the batches that shaped its lineage.

The benchmark evaluator supplies the optimization observations stored in the cache and, after selection, an independent frozen test.

2.2 SEPARATING PROPOSAL AND ADMISSION EVIDENCE

For a child proposed from parent s on batch P , its proposal history combines $\mathcal{B}_s$ with the current batch:

$$X(s, P) = P \cup \bigcup_{a \in \text{Anc}(s) \cup \{s\}} P_a^{\text{birth}} \quad (1)$$

A legal admission batch satisfies $A \cap X(s, P) = \emptyset$. For example, suppose the three diagnostic cases help produce p_1 from p_0 , and p_2 is later revised from p_1 using another batch. The original cases remain in p_2 ’s inherited proposal history. Admission for both p_1 and p_2 therefore draws outside the history that shaped each candidate, while the proposal traces remain available for diagnosis.

2.3 CHOOSING THE NEXT SEARCH ACTIONS

Let $\mathcal{H}_t$ contain the lineage, proposal batches, committed cache, remaining budget, and persistent routing state. A routing action $a_t = (\mathcal{A}_t, \mathbf{z}_t, Y_t)$ chooses active candidates, proposal parents, and a one-to-one assignment of their proposal slots to batches. Relative credit selects candidates; matching assigns batches using their recorded score gaps. Each fixed child receives a separately sampled admission batch outside its proposal history. The frozen benchmark test is reserved for final evaluation.

A feasible router enforces this exclusion, the wave-level stopping rule, one cached record per candidate–instance pair, and a separate final test. Its optimal policy minimizes expected simple regret for the returned program.

3 COMPASS: CANDIDATE SELECTION, MATCHING, AND ADMISSION

Each wave selects proposal parents and assigns them batches using the current cache. The resulting traces guide revisions, which are evaluated on legal admission batches with fixed references. Accepted outcomes update the comparison state and guide the next wave. The following sections define the credit, parent selection, matching, and admission steps.

3.1 RELATIVE CREDIT FROM SHARED OBSERVATIONS

Per-instance credit captures distinctions among candidates on the same instance. For example, if three candidates all succeed or all fail on a binary-scored instance, each receives zero credit. A single success gives credits $(1, -\frac{1}{2}, -\frac{1}{2})$; two successes give $(\frac{1}{2}, \frac{1}{2}, -1)$, with successful candidates listed first. The following definition extends this comparison to every legally observed frontier.

Let $\mathcal{F}_t(x)$ be the *frontier*: the candidates with the highest cached score among those for which $x \in \Omega_t(s)$, including all ties. In the all-failure example above, every candidate is on the frontier and each receives zero per-instance credit. Define the legally observed comparison set and the number of candidates on the frontier by

$$\mathcal{C}_t(x) = \{u : x \in \Omega_t(u)\}, \quad N_x = |\mathcal{C}_t(x)|, \quad y_{sx} = \mathbf{1}\{s \in \mathcal{F}_t(x)\}, \quad K_x = \sum_{u \in \mathcal{C}_t(x)} y_{ux}.$$

When $N_x \geq 2$, candidate s receives the per-instance credit

$$r_{sx} = y_{sx} - \frac{K_x - y_{sx}}{N_x - 1} = \frac{N_x y_{sx} - K_x}{N_x - 1}.$$

Only comparable exposures enter the aggregate:

$$\begin{aligned} \Omega_t^{\text{cmp}}(s) &= \{x \in \Omega_t(s) : N_x \geq 2\}, \quad E_s^{\text{cmp}} = |\Omega_t^{\text{cmp}}(s)|, \\ R_s &= \sum_{x \in \Omega_t^{\text{cmp}}(s)} r_{sx}, \quad G_s = \frac{R_s}{E_s^{\text{cmp}}}. \end{aligned} \tag{2}$$

The relative credit G_s is the average per-instance credit. It records how often s is on the current frontier relative to the other legally observed candidates on the same instances. An observation with $N_x = 1$ remains in the committed table but contributes no relative credit until another candidate is legally observed on x . Missing entries remain missing, and G_s is undefined when $E_s^{\text{cmp}} = 0$ rather than being set to zero. Whenever a commit changes an observation or a frontier, COMPASS recomputes N_x , K_x , all affected per-instance credits, and the resulting G_s values.

3.2 A CREDIT-RANKED ACTIVE SET

Let $\mathcal{R}_t = \{s : E_s^{\text{cmp}} > 0\}$ be the candidates with defined credit. Parents and descendants compete in this same set. Let $\rho \in \mathbb{N}_+$ be the active-rank cutoff. If $|\mathcal{R}_t| \geq \rho$, let g_ρ be the ρ -th largest G_s and retain every candidate with $G_s \geq g_\rho$; otherwise retain all of $\mathcal{R}_t$. The active set $\mathcal{A}_t$ can therefore contain more than ρ programs at a boundary tie, and a parent and its descendant may remain active simultaneously. If no candidate has defined credit, COMPASS falls back to the seed.

Proposal parents are taken in order from the active set, sorted by decreasing G_s with earlier candidate index breaking exact ties. Selection continues from the saved position across waves, returning to the start after the last candidate. For a fixed order (a, b, c) and wave width $w = 2$, successive waves draw (a, b) , (c, a) , and (b, c) . In general, each of the $w \in \mathbb{N}_+$ slots denotes one proposal attempt, with z_i its parent. When $|\mathcal{A}_t| < w$, a parent can receive multiple slots in the same wave.

3.3 TURNING CREDIT INTO PROPOSAL WORK AND LEGAL ADMISSION

Relative credit G_s selects which candidates to revise; matching determines which proposal batches they receive. COMPASS draws w proposal batches and matches them one-to-one to the w proposal slots. Proposal observations remain available for diagnosis, so matching uses the full cache, including

observations excluded from the parent’s credit. The matching weight $H(s, P)$ averages s ’s gap to the highest cached score among active candidates, computed separately for each instance in P :

$$H(s, P) = \frac{1}{|P|} \sum_{x \in P} \begin{cases} [\max_{r \in \mathcal{A}_t: q_t(r, x) \text{ finite}} q_t(r, x) - q_t(s, x)]_+, & q_t(s, x) \text{ finite,} \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

Here $[v]_+ = \max\{v, 0\}$. Missing parent observations contribute zero to this matching weight and remain missing in the cache. Let $\mathcal{I}_w = \{0, \dots, w-1\}$ index the drawn slots z_i and proposal minibatches P_j . COMPASS solves the maximum-weight matching, written as the binary program

$$\begin{aligned} Y^* \in \arg \max_{Y \in \{0,1\}^{w \times w}} \quad & \sum_{i,j \in \mathcal{I}_w} H(z_i, P_j) y_{ij} \\ \text{s.t.} \quad & \sum_{j \in \mathcal{I}_w} y_{ij} = 1 \quad \forall i, \quad \sum_{i \in \mathcal{I}_w} y_{ij} = 1 \quad \forall j. \end{aligned} \quad (4)$$

Thus distinct slots receive distinct proposal batches even when two slots carry the same parent label. The assignment maximizes the sum of their matching weights.

Proposal evaluation updates the existing parent’s cached observations. Tasks with usable execution traces and reflection data proceed to child generation. When perfect-score skipping is enabled, a task also requires at least one proposal score below the configured maximum. Write $m \leq w$ for the number of these remaining tasks, with parent s_i and proposal batch P_i . For each task independently, the benchmark evaluator samples an admission batch A_i satisfying

$$A_i \cap X(s_i, P_i) = \emptyset.$$

The evaluator freezes A_i and, for each $x \in A_i$, selects a reference from the current frontier with defined G , preferring the largest G and using seeded randomness for an exact tie. If no frontier candidate has defined credit, the sampled parent s_i is the reference. Denote this choice by $r_i(x)$. Missing reference observations are evaluated before the child is generated and stored using the highest-score cache rule. The admission instances, references, and outcomes remain hidden from the generator.

After the generator fixes child p_i from (s_i, P_i) , the evaluator scores it on A_i and compares its scores with the frozen reference scores. In this gate, $q(p_i, x)$ is the child’s new score and $q(r_i(x), x)$ is the fixed reference score on the same instance. The paired aggregate gate is

$$\Delta_i = \sum_{x \in A_i} [q(p_i, x) - q(r_i(x), x)], \quad \text{admit}(p_i) \iff \Delta_i > 0. \quad (5)$$

Proposal and reference evaluations update existing programs’ cached scores. Only credit-eligible entries affect their credit. An accepted child’s admission observations enter the cache with that child. Its proposal batch joins the inherited proposal history, and later observations outside that set can also contribute to its credit. COMPASS then recomputes every affected G_s , the tie-inclusive active set, and the queue. At the budget boundary, COMPASS selects the program with the largest defined G_s , with earliest index breaking an exact tie. The selected program then receives independent benchmark evaluation. Expanded execution details appear in Section B.

4 HOW COMPASS TURNS EVIDENCE INTO SEARCH

To see how the components fit together, follow one revision through the search loop. A proposal batch reveals a parent’s failures and shapes a child. Since those examples helped create the child, different evidence must establish that the revision is useful. The admission outcomes are compared with other candidates on the same instances to update their relative credit, which selects parents for the next proposal attempts. The formal statements below support three intuitions needed to read this loop: admission tests a fixed revision, credit records relative performance on shared instances, and a fixed rollout budget supports both candidate generation and evaluation.

4.1 A REVISION IS JUDGED BY PAIRED EVALUATION

Admission asks whether a fixed revision improves the total score on its evaluation batch. The child and its references are compared on the same instances. For example, paired differences of $(+1, +1, -1)$ sum to $+1$, so the child is admitted.

Proposition 1 (Reusing proposal examples can overstate performance). *For every admission size $n \geq 1$ and $\epsilon > 0$, there is a task distribution and a proposal rule that observes n examples, constructs a program, and scores one when those examples are reused for evaluation, although the program’s population utility is at most ϵ .*

A descendant can inherit behavior adapted to an ancestor’s proposal examples. COMPASS therefore carries the exclusion forward along the lineage. The construction in Section C illustrates this with memorization.

Paired evaluation. The proposal batch guides revision of the parent. The evaluator samples the admission batch outside $X(s, P)$, fixes its per-instance references, and hides this plan from the generator. Once the child is fixed, the gate in Equation (5) compares it with those references on the same instances. Theorem 1 establishes lineage exclusion and bounds the deviation of the mean paired difference from the average conditional mean.

4.2 ADMISSION OUTCOMES BECOME RELATIVE CREDIT

Admission decides whether a particular child enters the pool. Routing asks a different question: which accepted candidates have shown distinctions worth pursuing further? COMPASS answers it by comparing candidates on the same instance before aggregating across the partial cache.

Proposition 2 (Instance-wise credit conservation). *For every instance x with $N_x \geq 2$, the per-instance credits defined immediately before Equation (2) satisfy*

$$\sum_{s \in \mathcal{C}_t(x)} r_{sx} = 0.$$

This conservation law assigns credit to differences between candidates. If all N_x compared candidates are on the frontier, the instance assigns zero per-instance credit to every candidate. If one candidate is the only one on the frontier, it receives credit of one and each of the other $N_x - 1$ candidates receives $-1/(N_x - 1)$. Thus being on the frontier receives more credit when it distinguishes a candidate from its observed peers.

New results can change the meaning of scores already in the cache. Suppose a scores one and b scores zero on a shared binary-scored instance; their credits are 1 and -1 . If c later also scores one there, a and c are both on the frontier, so a ’s credit becomes $\frac{1}{2}$ with the same cached score. COMPASS recomputes the affected per-instance credits and G_s values as these comparisons change. An instance with one legal observation begins contributing relative credit when a second candidate becomes comparable there.

4.3 CREDIT DIRECTS THE NEXT WAVE

Matching accounts for parents competing for the same proposal batch.

For illustration, suppose two parent slots have the following matching weights:

	P_1	P_2
a	0.40	0.30
b	0.35	0.05

Both prefer P_1 . Assigning P_1 to a leaves P_2 for b , for a total weight of 0.45. Assigning P_2 to a and P_1 to b gives 0.65. Maximum-weight matching accounts for these competing preferences across the whole wave. After paired admission, accepted outcomes update the credit used in the next wave. At the budget boundary, the program with maximal defined G_s is returned.

4.4 SEARCH AND FINAL SELECTION

A search can miss a strong program or discover one and return a weaker candidate. The search gap and selection gap separate these two sources of regret. Let $\mathcal{P}_B$ be the candidates available when search stops and let $\hat{p}_B$ be the program selected by the largest defined G_s . Its regret has the exact

decomposition

$$R^* - R(\hat{p}_B) = \underbrace{R^* - \max_{p \in \mathcal{P}_B} R(p)}_{\text{search gap}} + \underbrace{\max_{p \in \mathcal{P}_B} R(p) - R(\hat{p}_B)}_{\text{selection gap}}. \quad (6)$$

Credit-ranked slots and proposal matching allocate candidate-generation work; paired admission determines which revisions enter the pool. To relate selection by relative credit to selection by sample mean, Proposition 3 first bounds the error of mean-based selection; Corollary 1 then connects it to the candidate chosen by G . Consider M candidates, each fixed before receiving n conditionally independent and identically distributed rewards under a common benchmark distribution, with mean $R(p_i)$. Let $\hat{R}_n(p_i)$ be the arithmetic mean of these rewards and let $\hat{p}_G$ maximize defined credit in this pool. Define

$$\xi_G = \max_{i \leq M} \hat{R}_n(p_i) - \hat{R}_n(\hat{p}_G). \quad (7)$$

Under these evaluation conditions, Corollary 1 gives, with probability at least $1 - \delta$,

$$R^* - R(\hat{p}_G) \leq \underbrace{R^* - \max_{i \leq M} R(p_i)}_{\text{search gap}} + \underbrace{2\sqrt{\frac{\log(2M/\delta)}{2n}}}_{\text{selection gap bound}} + \xi_G. \quad (8)$$

The term ξ_G is the difference in sample mean between the highest-mean candidate and the candidate selected by G . With complete shared binary observations and one score per candidate–instance pair, the two rankings coincide and $\xi_G = 0$ (Equation (11)).

5 EXPERIMENTS

5.1 EVALUATION QUESTIONS

The experiments evaluate whether provenance-constrained routing improves held-out benchmark utility under a fixed budget and whether the result persists across models and tasks.

5.2 TASKS, MODELS, AND EVALUATION

The study spans AIME-2025 and LiveBench-Math mathematical reasoning, IFBench instruction following with unseen constraint types, HotPotQA question answering, HoVer multi-hop fact verification, PUPA privacy-conscious delegation (Pyatkin et al., 2025; Balunović et al., 2025; White et al., 2025; Yang et al., 2018; Jiang et al., 2020; Li et al., 2025). The models are Qwen3-8B (Yang et al., 2025) and GPT-5.4-nano. PUPA retains its official GPT-4.1-mini judge for response quality and leakage of personally identifiable information (PII). This judge remains fixed throughout optimization and testing.

Official evaluation is separated from optimization. The Qwen3-8B AIME variant is evaluated on all 150 benchmark problems. HoVer uses 150/300/300 train, validation, and test instances with a 7,051-call budget; LiveBench-Math uses its frozen 126-problem test with a 1,839-call budget. Candidate selection uses optimization data, followed by one evaluation on the held-out test split.

5.3 QWEN3-8B: COMPARISON ACROSS SIX TASKS

Table 1 compares Qwen3-8B evaluations on all six shared tasks. Seed, GRPO, MIPROv2, GEPA, and GEPA+Merge reuse published GEPA-study task scores. We compare two COMPASS ablations: *w/o Credit & Matching* selects top-ranked candidates using frontier frequency over legal exposures (F/E) and omits skill-proposal matching; *w/o Matching* retains ranking by G_s in Equation (2) and omits only matching in Equation (4). The COMPASS row reports the program selected for each task. COMPASS has the highest six-task aggregate and the highest score on all six tasks, including PUPA, where it exceeds GEPA by 1.19 percentage points.

Table 1: **Qwen3-8B evaluation across all shared tasks (%)**. Each cell reports the official task aggregate. Aggregate is the unweighted mean over the six tasks, and Δ Seed is the aggregate improvement in percentage points. Both are computed from the displayed task scores before rounding to two decimal places. The first five rows reuse GEPA-study task scores. Ablation names indicate the removed components: Credit denotes G_s ranking, and Matching denotes skill-proposal matching. TextGrad and Trace were not reported for Qwen3-8B.

Method	HotPotQA	IFBench	HoVer	PUPA	AIME-2025	LiveBench-Math	Aggregate	Δ Seed
Seed	42.33	36.90	35.33	80.82	27.33	48.70	45.24	–
GRPO	43.33	35.88	38.67	86.66	38.00	51.26	48.97	+3.73
MIPROv2	55.33	36.22	47.33	81.55	20.00	46.60	47.84	+2.60
GEPA	62.33	38.61	52.33	91.85	32.00	51.95	54.85	+9.61
GEPA+Merge	64.33	28.23	51.67	86.26	32.00	51.95	52.41	+7.17
w/o Credit & Matching	64.67	40.31	51.67	86.91	57.33	66.67	61.26	+16.03
w/o Matching	65.33	40.82	50.67	88.85	60.67	67.68	62.34	+17.10
COMPASS	67.00	43.37	54.00	93.04	68.67	73.96	66.67	+21.44

The ablations separate the effects of credit ranking and matching. *w/o Credit & Matching*, *w/o Matching*, and COMPASS attain aggregate scores of 61.26%, 62.34%, and 66.67%, respectively. Comparing the two ablations evaluates the ranking signal; comparing *w/o Matching* with COMPASS evaluates the addition of skill-proposal matching.

The six tests contain 300, 294, 300, 221, 150, and 126 instances for HotPotQA, IFBench, HoVer, PUPA, AIME-2025, and LiveBench-Math, respectively. Candidate selection precedes the official test. The selected-candidate records are listed in Section D.

5.4 GPT-5.4-NANO: SAME-MODEL COMPARISON

Table 2 reports the GPT-5.4-nano comparison. Both task execution and reflection use GPT-5.4-nano. PUPA keeps the fixed benchmark scorer described above. COMPASS has the highest six-task aggregate and the highest score on four tasks. MIPROv2 is higher on HotPotQA, and GEPA is higher on HoVer.

Table 2: **GPT-5.4-nano evaluation across all shared tasks (%)**. Each cell reports the official task aggregate. Aggregate is the unweighted mean over the six tasks, and Δ Seed is the aggregate improvement in percentage points. The table includes methods with completed same-model evaluations.

Method	HotPotQA	IFBench	HoVer	PUPA	AIME-2025	LiveBench-Math	Aggregate	Δ Seed
Seed	31.33	48.98	45.33	74.86	15.33	35.09	41.82	–
MIPROv2	61.67	47.79	53.00	75.45	30.00	43.66	51.93	+10.11
GEPA	57.33	48.30	61.33	81.09	34.00	35.03	52.85	+11.03
COMPASS	60.00	51.36	60.33	84.14	42.00	45.03	57.14	+15.32

6 RELATED WORK

Reflective textual-program optimization. Reflexion and Self-Refine use language feedback to revise behavior, while TextGrad, Trace/OptoPrime, MIPROv2, and GEPA optimize textual components from execution evidence (Shinn et al., 2023; Madaan et al., 2023; Yuksekogonul et al., 2025; Cheng et al., 2024; Opsahl-Ong et al., 2024; Agrawal et al., 2025). COMPASS uses the resulting scores to choose which candidates to revise next and which batches to assign them.

Evolution, illumination, and skill search. Evolutionary prompt and skill methods maintain multiple candidates to avoid a single greedy trajectory (Agrawal et al., 2025; Mi et al., 2026; Moll et al., 2026; Zhang et al., 2026; Wang et al., 2026b). COMPASS keeps parents and descendants in one credit-ranked candidate pool. Its active set contains the highest-credit candidates, including all ties at the cutoff.

Adaptive evaluation and budget allocation. TRIPLE, POES, and Bayesian-Agent allocate evaluation or retrieval effort under limited budgets (Shi et al., 2024; Ma et al., 2026; Wu et al., 2026); TextBO and APEX optimize evaluation efficiency and development-example choice (Kang & Yoganarasimhan, 2026; Wang et al., 2026a). COMPASS allocates proposal attempts by relative credit and assigns their batches jointly by maximum-weight matching. Both decisions use the existing execution records.

Generalization after adaptive search. Repeated adaptive use of optimization data can widen the gap between selected validation performance and held-out test performance (Dwork et al., 2015). GEPA addresses this empirically through reusable rules and branch diversity (Agrawal et al., 2025). COMPASS carries proposal history through the lineage: examples used to create an ancestor remain excluded from a descendant’s admission. Held-out benchmark tests evaluate the selected program.

7 CONCLUSION

COMPASS uses execution results to decide which candidates to develop and which batches they should work on. Proposal history determines the eligible comparisons, and new results update the relative credit of candidates already in the pool. Relative credit selects proposal parents, and maximum-weight matching assigns their batches. Paired admission evaluates the resulting revisions on separate examples. Under the reported budgets, COMPASS achieves the highest six-task aggregate for both Qwen3-8B and GPT-5.4-nano, and leads on all six Qwen3-8B tasks.

REFERENCES

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning, 2025. URL <https://arxiv.org/abs/2507.19457>. ICLR 2026 Oral.
- Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Aime-2025, February 2025. URL https://huggingface.co/datasets/MathArena/aime_2025. Dataset from MathArena: Evaluating LLMs on Uncontaminated Math Competitions.
- Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms, 2024. URL <https://arxiv.org/abs/2406.16218>.
- Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Generalization in adaptive data analysis and holdout reuse. In *Advances in Neural Information Processing Systems*, volume 28, 2015. URL <https://proceedings.neurips.cc/paper/2015/hash/bad5f33780c42f2588878a9d07405083-Abstract.html>.
- Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. HoVer: A dataset for many-hop fact extraction and claim verification. In Trevor Cohn, Yulan He, and Yang Liu (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 3441–3460, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.309. URL <https://aclanthology.org/2020.findings-emnlp.309/>.
- Enoch Hyunwook Kang and Hema Yoganarasimhan. TextBO: Bayesian optimization in language space for eval-efficient self-improving AI. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2511.12063>.
- Siyan Li, Vethavikashini Chithrara Raghuram, Omar Khattab, Julia Hirschberg, and Zhou Yu. PA-PILLON: Privacy preservation from Internet-based and local language model ensembles. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 3371–3390, Albuquerque, New Mexico, April 2025. Association

for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.173. URL <https://aclanthology.org/2025.naacl-long.173/>.

Xiaoyu Ma, Yiwen Li, Haoyue Liu, Zhichao Wang, Ye Chen, Yongxin Guo, and Xiaoying Tang. Select smarter, not more: Prompt-aware evaluation scheduling with submodular guarantees, 2026. URL <https://arxiv.org/abs/2604.11328>.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In A. Oh, T. Nauermann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 46534–46594. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/91eddf07232fblb55a505a9e9f6c0ff3-Paper-Conference.pdf.

Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-Pro: Learning reusable skills from experience via non-parametric PPO for LLM agents, 2026. URL <https://arxiv.org/abs/2602.01869>. ICML 2026 Spotlight.

Johannes Moll, Jean-Philippe Corbeil, Jiazhen Pan, Martin Hadamitzky, Daniel Rueckert, Lisa Adams, and Keno Bressem. GRASP: Gated regression-aware skill proposer for self-improving LLM agents, 2026. URL <https://arxiv.org/abs/2605.29668>.

Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 9340–9366, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.525. URL <https://aclanthology.org/2024.emnlp-main.525/>.

Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following, 2025. URL <https://arxiv.org/abs/2507.02833>.

Chengshuai Shi, Kun Yang, Zihan Chen, Jundong Li, Jing Yang, and Cong Shen. Efficient prompt optimization through the lens of best arm identification. In *Advances in Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=FLNnlFBGMO>.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS ’23*, Red Hook, NY, USA, 2023. Curran Associates Inc.

Fei Wang, Si Si, Cho-Jui Hsieh, and Inderjit S. Dhillon. APEX: Automated prompt engineering expert with dynamic data selection, 2026a. URL <https://arxiv.org/abs/2606.11459>.

Zefeng Wang, Minxi Yan, Jinhe Bi, Sikuan Yan, Volker Tresp, and Yunpu Ma. MetaSkill-Evolve: Recursive self-improvement of LLM agents via two-timescale meta-skill evolution, 2026b. URL <https://arxiv.org/abs/2607.05297>.

Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddhartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-limited llm benchmark, 2025. URL <https://arxiv.org/abs/2406.19314>.

Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin, Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin Xu, Jia Li, and Jian Guo. Bayesian-Agent: Posterior-guided skill evolution for LLM agent harnesses, 2026. URL <https://arxiv.org/abs/2606.08348>.

540 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang
541 Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu,
542 Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin
543 Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang,
544 Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui
545 Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang
546 Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger
547 Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan
548 Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

549 Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov,
550 and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question
551 answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

552 Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin,
553 and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*,
554 639:609–616, 2025.

555 Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou,
556 Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S.
557 Yu. CoEvoSkills: Self-evolving agent skills via co-evolutionary verification, 2026. URL
558 <https://arxiv.org/abs/2604.01687>.

559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593

A NOTATION AND EXACT STATE

A.1 ELIGIBLE EXPOSURE AND FRONTIERS

At a fixed routing state t , reuse the credit-eligible set $\Omega_t(s) = \{x : q_t(s, x) \text{ exists and } x \notin \mathcal{B}_s\}$ from Section 2. The frontier is

$$\mathcal{F}_t(x) = \arg \max_{s: x \in \Omega_t(s)} q_t(s, x),$$

including all ties, even when all observed scores are zero. For each instance, define

$$\mathcal{C}_t(x) = \{u : x \in \Omega_t(u)\}, \quad N_x = |\mathcal{C}_t(x)|, \quad y_{sx} = \mathbf{1}\{s \in \mathcal{F}_t(x)\}, \quad K_x = \sum_{u \in \mathcal{C}_t(x)} y_{ux}.$$

When $N_x \geq 2$, the per-instance credit is

$$r_{sx} = y_{sx} - \frac{K_x - y_{sx}}{N_x - 1}.$$

The relative credit G_s averages these per-instance credits:

$$\Omega_t^{\text{cmp}}(s) = \{x \in \Omega_t(s) : N_x \geq 2\}, \quad E_s^{\text{cmp}} = |\Omega_t^{\text{cmp}}(s)|, \quad R_s = \sum_{x \in \Omega_t^{\text{cmp}}(s)} r_{sx}, \quad G_s = R_s / E_s^{\text{cmp}}.$$

An $N_x = 1$ observation remains committed but does not enter the aggregate. Whenever a new finite observation changes $\mathcal{C}_t(x)$ or $\mathcal{F}_t(x)$, the router recomputes N_x , K_x , and every affected candidate contribution on that instance. Candidates with $E_s^{\text{cmp}} = 0$ have undefined credit.

A.2 INHERITED PROPOSAL HISTORY

If parent s is proposed on batch P , the prospective child cannot use its current proposal instances or any ancestor birth-proposal instances as admission evidence. Let P_a^{birth} be the recorded proposal batch that created accepted candidate a , with $P_0^{\text{birth}} = \emptyset$ for the seed. The child’s proposal history is

$$X(s, P) = P \cup \bigcup_{a \in \text{Anc}(s) \cup \{s\}} P_a^{\text{birth}}.$$

Proposal history records the instances used in a candidate’s own and its ancestors’ proposal batches. Admission observations retain their role as evaluation evidence. The proposal history $\mathcal{B}_s$ governs which committed observations may contribute to candidate s ’s relative credit; $X(s, P)$ extends that exclusion to the prospective child’s admission.

A.3 CHECKPOINT STATE

The optimizer checkpoint records the accepted programs, parent relation, committed reward cache, frontier map, logical call count, active-queue cursor, proposal assignments, sampled admission batches, and seeded random states. The queue cursor is interpreted modulo the current active order after every recomputation.

B ALGORITHMIC DETAILS

B.1 PROPOSAL ALLOCATION IMPLEMENTATION

Equations (3) and (4) define the proposal objective. The implementation builds the complete slot–batch weight matrix from the committed cache and solves the resulting maximum-weight assignment exactly. Duplicate parent labels remain distinct left-side slot vertices. Missing cache entries remain unchanged during assignment.

B.2 DIRECT LEGAL ADMISSION IMPLEMENTATION

For each of the $m \leq w$ remaining proposal tasks, the benchmark evaluator samples one batch A_i from that task’s admission sampler after removing every instance in $X(s_i, P_i)$. For each instance in the sampled batch, the evaluator selects the frontier candidate with defined, maximal G as reference, using seeded randomness for an exact tie and the sampled parent when no frontier candidate has defined credit. The resulting plan is hidden from the generator, required missing reference observations are filled on the existing reference programs, and the fixed child is admitted exactly when the paired aggregate in Equation (5) is positive.

B.3 WAVE EXECUTION

Algorithm 1 One COMPASS wave

Require: committed routing state $\mathcal{H}_t$, wave width w , active cutoff ρ
Ensure: updated committed state $\mathcal{H}_{t+1}$

- 1: Recompute every defined G_s from the committed cache
- 2: $\mathcal{A}_t \leftarrow$ tie-inclusive Top- ρ candidates, sorted by G_s and candidate index
- 3: If $\mathcal{A}_t = \emptyset$, set $\mathcal{A}_t \leftarrow \{0\}$ ▷ Seed fallback
- 4: $(z_i)_{i \in \mathcal{I}_w} \leftarrow$ next w parents from $\mathcal{A}_t$, continuing from the saved position
- 5: Draw $(P_j)_{j \in \mathcal{I}_w}$ and form $M_{ij} \leftarrow H(z_i, P_j)$
- 6: $Y^* \leftarrow \text{MAXWEIGHTASSIGNMENT}(M)$
- 7: **for all** assigned parent–proposal pairs (s_i, P_i) **in parallel do**
- 8: Evaluate s_i on P_i and construct its reflection data
- 9: Update the cache with new pairs or strictly higher parent scores
- 10: **end for**
- 11: $\mathcal{T} \leftarrow$ tasks with usable traces and reflection data, after configured perfect-score skipping
- 12: $m \leftarrow |\mathcal{T}| \leq w$
- 13: **for all** $(s_i, P_i) \in \mathcal{T}$ **in parallel do**
- 14: $A_i \leftarrow \text{SAMPLEADMISSION}(s_i, P_i)$ subject to $A_i \cap X(s_i, P_i) = \emptyset$
- 15: $r_i \leftarrow \text{BINDREFERENCES}(A_i)$; freeze and hide (A_i, r_i) from the generator
- 16: Evaluate missing references and store their scores and outputs
- 17: $p_i \leftarrow \text{REFLECT}(s_i, P_i)$; evaluate p_i on A_i
- 18: $\Delta_i \leftarrow \sum_{x \in A_i} [q(p_i, x) - q(r_i(x), x)]$
- 19: **if** $\Delta_i > 0$ **then**
- 20: Commit p_i and its lineage-eligible admission evidence
- 21: **end if**
- 22: **end for**
- 23: Update the logical budget, frontiers, credits, and queue cursor; persist $\mathcal{H}_{t+1}$

Repeat Algorithm 1 while the cumulative logical call count is below B , and complete the final wave before stopping. Report both the target B and actual calls. Request-level retries add no logical task result.

C PROPERTIES OF ADMISSION, CREDIT, AND SELECTION

We prove the exclusion and paired-concentration properties of admission, instance-wise credit conservation, finite-pool selection bounds, coverage of the active set, and optimality of the proposal assignment.

C.1 WHY PROPOSAL AND ADMISSION EVIDENCE ARE SEPARATE

Proof of Proposition 1. Let the task distribution be uniform on a domain of size $K \geq n/\epsilon$. After observing $S = (x_1, \dots, x_n)$, let the proposal rule construct a program whose reward is one exactly on the distinct elements appearing in S and zero elsewhere. Every reused evaluation example belongs to that set, so the empirical reward on S is one. The population reward is the fraction of the domain covered by the observed set, at most $n/K \leq \epsilon$. $\square$

The construction motivates admission on examples outside those that shaped the candidate. Recording proposal history extends this separation through the lineage, because a descendant can retain behavior adapted to an ancestor’s proposal batch.

C.2 PAIRED ADMISSION

Theorem 1 (Conditional concentration for paired admission). *Let $\mathcal{V}$ be the information visible to the generator when child p is fixed from parent s and proposal batch P , and suppose that p is $\mathcal{V}$ -measurable. Before generation, the evaluator fixes an admission batch $A = (X_1, \dots, X_n)$, with $n \geq 1$, outside $X(s, P)$ and binds each reference $r(X_j)$ from the committed pre-child state and seeded tie-breaking randomness. Admission identities, references, and outcomes are not visible in $\mathcal{V}$. Then*

$$A \cap P = \emptyset, \quad A \cap P_a^{\text{birth}} = \emptyset \quad \text{for every } a \in \text{Anc}(s) \cup \{s\}.$$

After p is fixed, reveal the paired outcomes sequentially. Let $(\mathcal{H}_j)_{j=0}^n$ be the filtration generated by the fixed child, the frozen admission plan, and the first j revealed paired outcomes. Define

$$Z_j = q(p, X_j) - q(r(X_j), X_j), \quad \mu_j = \mathbb{E}[Z_j \mid \mathcal{H}_{j-1}],$$

$$\hat{\mu}_A(p; r) = \frac{1}{n} \sum_{j=1}^n Z_j, \quad \bar{\mu}_A(p; r) = \frac{1}{n} \sum_{j=1}^n \mu_j.$$

Because $Z_j \in [-1, 1]$, for every $\delta \in (0, 1)$, with probability at least $1 - \delta$,

$$|\hat{\mu}_A(p; r) - \bar{\mu}_A(p; r)| \leq \sqrt{\frac{2 \log(2/\delta)}{n}}. \quad (9)$$

For the i th task, the implemented gate is equivalently admit(p_i) $\iff \hat{\mu}_{A_i}(p_i; r_i) > 0$.

Proof of Theorem 1. Because $X(s, P)$ contains both P and every P_a^{birth} for $a \in \text{Anc}(s) \cup \{s\}$, legality $A \cap X(s, P) = \emptyset$ gives the displayed disjointness relations. After the child is fixed, each paired difference Z_j lies in an interval of length two. Conditional Hoeffding–Azuma applied to $Z_j - \mathbb{E}[Z_j \mid \mathcal{H}_{j-1}]$ gives

$$\Pr(|\hat{\mu}_A(p; r) - \bar{\mu}_A(p; r)| \geq \epsilon) \leq 2 \exp\left(-\frac{n\epsilon^2}{2}\right).$$

Setting the right-hand side to δ proves Equation (9). $\square$

C.3 PER-INSTANCE CREDIT SUMS TO ZERO

Proof of Proposition 2. For a fixed comparable instance x , use $\sum_{s \in \mathcal{C}_t(x)} y_{sx} = K_x$ to obtain

$$\begin{aligned} \sum_{s \in \mathcal{C}_t(x)} r_{sx} &= K_x - \frac{\sum_{s \in \mathcal{C}_t(x)} (K_x - y_{sx})}{N_x - 1} \\ &= K_x - \frac{N_x K_x - K_x}{N_x - 1} = 0. \end{aligned}$$

The statement is per instance; the candidate averages G_s need not sum to zero because their comparable exposure sets and denominators can differ. $\square$

C.4 FINITE-POOL SELECTION

Proposition 3 (Finite-pool mean selection). *For any $M, n \in \mathbb{N}_+$ and $\delta \in (0, 1)$, let $p_1, \dots, p_M$ be adaptively generated candidates. Let $\mathcal{J}_{i-1}$ contain the history before candidate p_i is evaluated, with p_i measurable with respect to that history. Conditional on $\mathcal{J}_{i-1}$, suppose p_i receives n independent and identically distributed rewards in $[0, 1]$ under a common benchmark evaluation distribution, with mean $R(p_i)$. Let $\hat{R}_n(p_i)$ be their arithmetic mean and define*

$$\tilde{p}_{M,n} \in \arg \max_{p \in \{p_1, \dots, p_M\}} \hat{R}_n(p).$$

Then, with probability at least $1 - \delta$,

$$R^* - R(\tilde{p}_{M,n}) \leq \underbrace{R^* - \max_{i \leq M} R(p_i)}_{\text{search gap}} + \underbrace{2\sqrt{\frac{\log(2M/\delta)}{2n}}}_{\text{selection noise}}. \quad (10)$$

Proof of Proposition 3. Conditional on $\mathcal{J}_{i-1}$, Hoeffding’s inequality gives

$$\Pr\left(|\hat{R}_n(p_i) - R(p_i)| > \epsilon \mid \mathcal{J}_{i-1}\right) \leq 2e^{-2n\epsilon^2}.$$

Taking expectations over the history and a union bound over the M candidates gives, with probability at least $1 - \delta$,

$$\max_{i \leq M} |\hat{R}_n(p_i) - R(p_i)| \leq \epsilon, \quad \epsilon = \sqrt{\frac{\log(2M/\delta)}{2n}}.$$

Let p_M^* maximize population utility in the pool. On this event,

$$R(p_M^*) - R(\tilde{p}_{M,n}) \leq \hat{R}_n(p_M^*) - \hat{R}_n(\tilde{p}_{M,n}) + 2\epsilon \leq 2\epsilon.$$

Adding the search gap proves Equation (10). $\square$

Taking expectations and bounding regret on the failure event by one also gives

$$\mathbb{E}[R^* - R(\tilde{p}_{M,n})] \leq \mathbb{E}\left[R^* - \max_{i \leq M} R(p_i)\right] + 2\sqrt{\frac{\log(2M/\delta)}{2n}} + \delta.$$

Along a nested candidate sequence, the search gap is nonincreasing in M ; the selection-noise term decreases with n for fixed M .

Selection by relative credit.

Corollary 1 (Credit-based selection). *Under the conditions of Proposition 3, let $\hat{p}_G$ be a candidate with the largest defined G in the pool and let ξ_G be as in Equation (7). Then Equation (8) holds with probability at least $1 - \delta$. Moreover,*

$$\mathbb{E}[R^* - R(\hat{p}_G)] \leq \mathbb{E}\left[R^* - \max_{i \leq M} R(p_i)\right] + 2\sqrt{\frac{\log(2M/\delta)}{2n}} + \mathbb{E}[\xi_G] + \delta.$$

Proof of Corollary 1. On the simultaneous concentration event used above,

$$\begin{aligned} R(p_M^*) - R(\hat{p}_G) &\leq \hat{R}_n(p_M^*) - \hat{R}_n(\hat{p}_G) + 2\epsilon \\ &\leq \xi_G + 2\epsilon. \end{aligned}$$

Adding the search gap yields Equation (8). The expectation bound follows because regret is at most one on the failure event and the search gap and ξ_G are nonnegative. $\square$

Coinciding rankings for binary scores. Suppose the $M \geq 2$ candidates share the same n legal instances, each candidate–instance pair has one binary score, and these are all their comparable observations. Thus $N_x = M$ and $\hat{R}_n(p_s) = n^{-1} \sum_x q(s, x)$. On each instance, either at least one score is one and $y_{sx} = q(s, x)$, or every score is zero and every candidate is on the frontier. In both cases,

$$r_{sx} = q(s, x) - \frac{1}{M-1} \sum_{u \neq s} q(u, x).$$

Averaging over the common instances gives

$$G_s = \frac{M}{M-1} \left(\hat{R}_n(p_s) - \frac{1}{M} \sum_{u=1}^M \hat{R}_n(p_u) \right). \quad (11)$$

This is a shared positive affine transformation of the sample means, so their rankings and ties coincide and $\xi_G = 0$.

C.5 STRUCTURAL PROPERTIES OF THE ROUTER

Proposition 4 (Proposal turns for a fixed active set). *For a fixed ordered active set of size m , every block of m consecutive queue draws contains each active candidate exactly once, including when a width- w wave wraps around the queue.*

Proof. Index the fixed active order by $0, \dots, m-1$ and let the persistent cursor be c . Consecutive draws are $(c+r) \bmod m$. For $r = 0, \dots, m-1$, these residues are pairwise distinct and cover every index. Grouping the same sequence into blocks of width w does not alter the argument, including a block that wraps around the end of the queue. $\square$

Proposition 5 (Maximum-weight proposal matching). *Conditional on the w drawn parent slots and w proposal minibatches, Equation (4) maximizes total matching weight over all one-to-one slot–minibatch assignments.*

Proof of maximum-weight proposal matching. The w parent slots and w minibatches define a complete bipartite graph with edge weight $H(z_i, P_j)$. A feasible binary matrix in Equation (4) is exactly a perfect matching, or equivalently a permutation matrix. Selecting the maximum-weight feasible matrix therefore returns the maximum-weight matching. Repeated parent labels do not change the graph because their slot vertices remain distinct. $\square$

D DETAILED EXPERIMENTAL PROTOCOL

D.1 METHOD CONFIGURATION

The method in Section 3 uses relative credit, applies the tie-inclusive Top- ρ cutoff to all candidates with defined credit, uses the exact proposal assignment in Equation (4), and samples admission batches independently for each remaining proposal task. The implementation names this credit `leave_one_out_frontier_credit`. The protocol evaluates the selected program once.

The Qwen3-8B runs select candidates 55, 78, 76, 34, 15, and 30 for HotPotQA, IFBench, HoVer, PUPA, AIME-2025, and LiveBench-Math, respectively. GEPA’s GPT-5.4-nano AIME-2025 result uses its first completed official evaluation. Each saved run records rollout counts, candidate indices, tokens, and costs. Its checkpoint retains lineage, queue, assignment, and cache updates for analysis of the search mechanism.

D.2 OFFICIAL EVALUATION PROTOCOL

Reported results use the candidate fixed at the end of optimization and a complete official evaluation. Each run records the selected program, source version, configuration, and model. IFBench requires 294 finite example scores whose mean equals the aggregate. AIME requires all 150 benchmark problems. HoVer requires 300 finite binary scores, and LiveBench-Math requires all 126 frozen test positions under its official aggregation.

Evaluation uses the selected candidate with its matching implementation and configuration. Service failures are recorded separately from benchmark scores. Preflight runs and intermediate checkpoints remain separate from the final evaluation, and the candidate is fixed before its test scores are observed.

D.3 RESOURCE ACCOUNTING

Logical task calls and final-wave overshoot are reported separately from physical language-model calls. The latter include input/output tokens and route-priced cost. A local task deadline can produce a benchmark failure score when that outcome is defined by the evaluator. Other provider, schema, serialization, and engineering errors terminate the evaluation batch. A resumable run must preserve the exact source, config, cache, checkpoint, and random-number-generator state.

E BROADER IMPACT

Evidence routing can reduce the number of expensive environment executions needed to discover a useful textual program and can make search decisions easier to audit. The same efficiency can also optimize undesirable procedures or exploit imperfect rewards. Responsible deployment calls for secure tool policies, sandboxing, red-team evaluation, and human review.

Use of large language models. Language models assisted with literature review, editing, code, and figure design. The authors verified every citation, algorithm, and artifact; model output was neither experimental evidence nor used to choose test candidates.